

JAVA

Java Arrays

FUNDAMENTOS

Do básico à manipulação de matrizes: como o Java guarda coleções de tamanho fixo na memória, os métodos utilitários de `java.util.Arrays` e os algoritmos de busca mais usados no dia a dia — tudo com os dados de uma estação meteorológica como fio condutor dos exemplos.

01 Introdução aos Arrays

02 Declarando e Inicializando Arrays

03 Arrays como Objetos em Java

04 O Enhanced For Loop (For-each)

05 A Classe `java.util.Arrays`

06 Busca em Arrays

07 Variable Arguments (Varargs)

08 Arrays Bidimensionais

09 Arrays Multidimensionais

JavaStudiesHub

MOTIVAÇÃO

01

Introdução aos Arrays

O problema que os arrays resolvem e a estrutura de dados mais fundamental do Java

O problema de guardar muitos valores do mesmo tipo

Imagine que você está construindo um sistema para uma estação meteorológica que registra a temperatura de hora em hora, ao longo de um dia inteiro — 24 leituras. Sem uma estrutura adequada, você precisaria declarar 24 variáveis separadas: `temperaturaHora0`, `temperaturaHora1`, `temperaturaHora2`... Além de trabalhoso, esse código não escala: e se a estação passar a coletar leituras a cada minuto, totalizando 1.440 variáveis?

Um **array** resolve exatamente esse problema: em vez de N variáveis separadas, ele cria uma única estrutura capaz de guardar N valores do mesmo tipo, todos acessíveis através de um único nome e de um índice numérico. É a estrutura de dados mais fundamental do Java — praticamente todo programa, cedo ou tarde, acaba usando um array em algum ponto, mesmo que indiretamente através de outras estruturas construídas sobre eles.

O que exatamente é um array

Um array é uma estrutura linear que guarda uma sequência de valores do mesmo tipo, dispostos de forma contígua na memória. É possível criar arrays de qualquer tipo primitivo (`int`, `double`, `boolean`, `char`) ou de qualquer classe (`String`, ou uma classe própria como `LeituraDeSensor`).

Cada posição do array é identificada por um índice numérico que sempre começa em 0. Em um array com 24 leituras, os índices válidos vão de 0 a 23 — tentar acessar o índice 24, ou qualquer índice negativo, dispara em tempo de execução um `ArrayIndexOutOfBoundsException`, um dos erros mais comuns entre quem está começando com a linguagem.

```
double[] temperaturas = new double[24]; // 24 posições, uma por hora
double[] leiturasDoDia = {18.5, 17.2, 16.8, 16.1, 19.4}; // já inicializado com 5 valores

System.out.println(leiturasDoDia[0]); // 18.5 – primeira leitura
System.out.println(leiturasDoDia[4]); // 19.4 – última leitura
System.out.println(leiturasDoDia.length); // 5 – tamanho do array
// leiturasDoDia[5] lançaria ArrayIndexOutOfBoundsException!
```

DICA

`length` devolve o número total de posições do array. O último índice válido é sempre `length - 1`, nunca `length` — um deslize fácil de cometer.

Declarando e Inicializando Arrays

As duas formas de criar um array, valores padrão e a propriedade length

Duas formas de criar um array

A primeira forma usa a palavra-chave **new**, informando apenas o tamanho desejado. Nesse caso, todas as posições recebem automaticamente o valor padrão do tipo: 0 para inteiros, 0.0 para tipos de ponto flutuante, false para boolean e null para qualquer tipo de referência.

A segunda forma é o **array initializer**: os valores são informados diretamente entre chaves no momento da declaração. Aqui o tamanho do array é deduzido automaticamente pela quantidade de valores fornecidos, e a palavra new sequer é necessária.

```
// Forma 1: new – cria com valores padrão
int[] contagemDeChuva = new int[7]; // [0, 0, 0, 0, 0, 0, 0]
String[] nomesDasEstacoes = new String[3]; // [null, null, null]
boolean[] alertasAtivos = new boolean[4]; // [false, false, false, false]

// Forma 2: array initializer – valores já definidos
double[] umidadeRelativa = {62.5, 58.0, 71.3, 65.9};
String[] estacoes = {"Estacao Norte", "Estacao Sul", "Estacao Central"};

// Array 2D com initializer aninhado:
int[][] chuvaPorMesEEstacao = {{12, 8, 20}, {5, 15, 9}};
```

A propriedade length

Todo array em Java expõe a propriedade **length**, que informa quantas posições ele tem. É importante notar que length é um campo, não um método — diferente de String.length(), que exige parênteses. Confundir os dois é um engano frequente para quem está começando.

```
double[] leituras = {21.0, 22.5, 19.8, 23.1, 20.4};
System.out.println(leituras.length); // correto: 5
// System.out.println(leituras.length()); // ERRO de compilação!

for (int i = 0; i < leituras.length; i++) {
    System.out.println("Hora " + i + ": " + leituras[i] + " graus");
}
```

DICA

Nunca fixe o tamanho do array dentro de um loop. Use sempre arr.length: se o array mudar de tamanho no futuro, o loop continua correto sem precisar de ajustes.

Arrays como Objetos em Java

Herança de Object, semântica de referência e como copiar corretamente

Arrays também são objetos

Em Java, praticamente tudo que não é primitivo é um objeto — e arrays não fogem à regra. Um array é uma espécie de classe especial que herda diretamente de `java.lang.Object`, o que dá a ele acesso a métodos como `toString()` e `equals()`. Só que as implementações padrão desses métodos raramente fazem o que se espera: `toString()` em um array retorna algo como `[D@4aa298b7`, o endereço do objeto — não os valores guardados nele.

Referência, não cópia

Quando uma variável de array é declarada, ela não guarda o array em si — ela guarda uma **referência**, um endereço que aponta para onde o array realmente vive no heap. Isso tem uma consequência importante: atribuir um array a outra variável não cria uma cópia; cria uma segunda variável apontando para o mesmo objeto. Qualquer alteração feita por uma delas fica visível através da outra.

```
double[] leituraOriginal = {18.0, 19.5, 20.1};
double[] leituraDuplicada = leituraOriginal; // aponta para o MESMO array!

leituraDuplicada[0] = 99.9;
System.out.println(leituraOriginal[0]); // 99.9 – o original também mudou!

// Para gerar uma cópia INDEPENDENTE, use Arrays.copyOf:
import java.util.Arrays;
double[] copiaSegura = Arrays.copyOf(leituraOriginal, leituraOriginal.length);
copiaSegura[0] = 10.0;
System.out.println(leituraOriginal[0]); // ainda 99.9 – não foi afetado
```

ATENÇÃO

Arrays de objetos têm dois níveis de indireção: a variável aponta para o array, e cada posição do array aponta para um objeto. Copiar o array copia apenas as referências — os objetos apontados continuam sendo compartilhados.

O Enhanced For Loop (For-each)

A sintaxe simplificada de percorrer um array e quando ela não é suficiente

O que é o for-each

O enhanced for loop, ou for-each, é uma forma simplificada de percorrer todos os elementos de um array (ou de qualquer coleção Iterable) sem precisar controlar um índice manualmente. Isso torna o código mais limpo e elimina erros clássicos de índice, como usar `<= length` em vez de `< length`.

A sintaxe usa dois-pontos como separador. A cada iteração, uma variável local recebe o valor da posição atual, percorrendo o array do primeiro ao último elemento, em ordem.

```
double[] leituras = {18.2, 19.5, 21.0, 17.8};

// For-each – direto ao ponto
for (double leitura : leituras) {
    System.out.println(leitura + " graus");
}

// Equivalente com for tradicional
for (int i = 0; i < leituras.length; i++) {
    System.out.println(leituras[i] + " graus");
}
```

Quando o for tradicional ainda é necessário

O for-each é ótimo quando o objetivo é apenas ler os valores, sem se importar com a posição. Mas existem cenários em que ele não serve: quando é preciso modificar os elementos do array (a variável do for-each é apenas uma cópia do valor lido), quando é preciso conhecer o índice atual, quando é preciso percorrer de trás para frente, ou quando dois arrays precisam ser percorridos em paralelo, usando o mesmo índice.

```
double[] temperaturas = {20.0, 21.0, 22.0};

// ERRADO: não altera o array original
for (double t : temperaturas) {
    t = t + 1; // altera apenas a cópia local
}
System.out.println(temperaturas[0]); // ainda 20.0

// CORRETO: use o for tradicional para modificar
for (int i = 0; i < temperaturas.length; i++) {
    temperaturas[i] = temperaturas[i] + 1;
}
System.out.println(temperaturas[0]); // agora 21.0
```

A Classe `java.util.Arrays`

`sort`, `fill`, `copyOf`, `toString` e `equals`

Uma classe só de métodos estáticos

`java.util.Arrays` reúne, em um único lugar, os métodos utilitários mais usados para trabalhar com arrays. Todos os seus métodos são `static` — chamados diretamente pela classe, sem nunca instanciá-la — e ela está disponível desde as versões mais antigas do Java. Basta importar `java.util.Arrays` no topo do arquivo para usá-la.

`sort`, `fill`, `copyOf` e `toString`

`Arrays.sort(array)` ordena o array em ordem crescente diretamente nele mesmo (in-place). Para tipos primitivos, o algoritmo usado é uma variante otimizada do Quicksort; para objetos, Timsort — em ambos os casos, com complexidade $O(n \log n)$ no caso médio.

`Arrays.fill(array, valor)` preenche todas as posições com um mesmo valor; existe também uma versão que recebe um intervalo [início, fim). `Arrays.copyOf(array, novoTamanho)` devolve um novo array com o tamanho pedido, preenchendo com o valor padrão qualquer posição extra. E `Arrays.toString(array)` transforma o array em uma `String` legível — sem ele, imprimir um array diretamente mostraria apenas o endereço de memória do objeto.

```
import java.util.Arrays;

double[] leituras = {23.4, 18.1, 25.0, 15.6, 30.2};
System.out.println(Arrays.toString(leituras)); // [23.4, 18.1, 25.0, 15.6, 30.2]

Arrays.sort(leituras);
System.out.println(Arrays.toString(leituras)); // [15.6, 18.1, 23.4, 25.0, 30.2]

Arrays.fill(leituras, 0.0);
System.out.println(Arrays.toString(leituras)); // [0.0, 0.0, 0.0, 0.0, 0.0]

double[] expandido = Arrays.copyOf(leituras, 8);
System.out.println(Arrays.toString(expandido)); // 8 posições, extras com 0.0

// Arrays.equals – compara valor a valor, diferente de ==
double[] a = {20.0, 21.0};
double[] b = {20.0, 21.0};
System.out.println(a == b); // false – referências diferentes
System.out.println(Arrays.equals(a, b)); // true – mesmos valores
```

Busca em Arrays

Busca linear, busca binária e Arrays.binarySearch

Busca linear

A busca linear percorre o array posição por posição, comparando cada elemento ao valor procurado. Sua complexidade é $O(n)$: no pior caso, todas as N posições são examinadas antes de encontrar o alvo, ou concluir que ele não existe no array. Para arrays pequenos, ou não ordenados, a busca linear é simples e perfeitamente adequada.

```
// Busca linear — funciona mesmo em array não ordenado
static int buscarLeituraExata(double[] leituras, double alvo) {
    for (int i = 0; i < leituras.length; i++) {
        if (leituras[i] == alvo) return i; // encontrou, devolve o índice
    }
    return -1; // não encontrado
}
```

Busca binária

A busca binária é bem mais eficiente — complexidade $O(\log n)$ — mas exige que o array já esteja ordenado. A lógica: compare o valor procurado com o elemento do meio; se for igual, encontrou; se for menor, repita a busca só na metade esquerda; se for maior, só na metade direita. A cada passo, o espaço de busca cai pela metade — em um array de 1 bilhão de elementos, no máximo 30 comparações resolvem a busca.

Arrays.binarySearch(array, valor) já implementa esse algoritmo pronto para uso. Ele devolve o índice do valor encontrado, ou um número negativo se ele não existir no array — número esse que também indica onde o valor deveria ser inserido para manter a ordenação.

```
import java.util.Arrays;

double[] leiturasOrdenadas = {15.6, 18.1, 20.0, 23.4, 25.0, 30.2};
// 0 1 2 3 4 5

int indice = Arrays.binarySearch(leiturasOrdenadas, 23.4); // devolve 3
int naoAchou = Arrays.binarySearch(leiturasOrdenadas, 19.0); // devolve negativo

System.out.println("Índice de 23.4: " + indice);
System.out.println("19.0 encontrado? " + (naoAchou >= 0));
```

ATENÇÃO

Arrays.binarySearch exige que o array esteja ordenado em ordem crescente antes da busca. Usá-lo em um array desordenado gera resultados imprevisíveis — chame Arrays.sort antes, se necessário.

Variable Arguments (Varargs)

A sintaxe ... para métodos que aceitam qualquer quantidade de argumentos

O que são varargs

Varargs é o recurso que permite declarar um método capaz de receber de zero a qualquer quantidade de argumentos do mesmo tipo. Por baixo dos panos, o Java trata o parâmetro vararg como um array comum: dá para percorrê-lo com for-each, checar seu length e acessar posições por índice normalmente.

Sintaxe e regras

A sintaxe usa três pontos após o tipo do parâmetro — por exemplo, void metodo(double... valores). Isso permite chamar o método sem argumento nenhum, com um único valor, com vários, ou até passando um array já pronto diretamente.

Algumas regras: só pode existir um parâmetro vararg por método; ele obrigatoriamente precisa ser o último da lista de parâmetros; e é possível combinar parâmetros normais antes dele.

```
// Sem varargs – obrigatório montar um array antes de chamar
static void registrarLeituras(double[] valores) {
    for (double v : valores) System.out.println(v);
}
registrarLeituras(new double[]{18.0, 19.5, 20.1}); // verboso

// Com varargs – chamada direta
static void registrarLeituras(double... valores) {
    for (double v : valores) System.out.println(v);
}
registrarLeituras(18.0, 19.5, 20.1); // simples
registrarLeituras(); // zero argumentos: também é válido

// Combinando parâmetro normal com vararg
static double calcularMedia(String estacao, double... leituras) {
    double soma = 0;
    for (double v : leituras) soma += v;
    return leituras.length == 0 ? 0 : soma / leituras.length;
}
System.out.println(calcularMedia("Estacao Norte", 18.0, 20.0, 22.0)); // 20.0
```


Arrays Bidimensionais

Linhas, colunas, inicialização e arrays com linhas de tamanhos diferentes

O que é um array 2D

Um array bidimensional é, na essência, um array de arrays — pode ser visualizado como uma tabela com linhas e colunas, parecida com uma planilha. Em Java, ele é declarado com dois pares de colchetes: `double[][]`, onde o primeiro representa as linhas e o segundo as colunas. É uma estrutura natural para representar uma grade de sensores, onde cada linha é uma estação e cada coluna é uma leitura horária.

Declaração, inicialização e acesso

Para acessar um elemento, usam-se dois índices: `grade[linha][coluna]`. Para percorrer todas as posições, usam-se dois loops aninhados — um para linha e outro para coluna — e o `for-each` aninhado funciona perfeitamente também sobre arrays 2D.

```
// Declaração com new — tudo zerado
double[][] grade = new double[3][24]; // 3 estações, 24 leituras horárias cada

// Declaração com initializer
double[][] leiturasPorEstacao = {
    {18.0, 19.0, 20.0}, // Estação Norte
    {22.0, 23.5, 21.0}, // Estação Sul
    {17.5, 18.2, 19.9} // Estação Central
};

System.out.println(leiturasPorEstacao[1][2]); // 21.0 (Estação Sul, 3ª leitura)
System.out.println(leiturasPorEstacao.length); // 3 (número de estações)
System.out.println(leiturasPorEstacao[0].length); // 3 (leituras da Estação Norte)

// Percorrendo com for-each aninhado
for (double[] estacao : leiturasPorEstacao) {
    for (double leitura : estacao) {
        System.out.print(leitura + " ");
    }
    System.out.println();
}
```

Jagged arrays — linhas de tamanhos diferentes

Em Java, cada linha de um array 2D é, na verdade, um array independente — o que significa que elas podem ter tamanhos diferentes entre si. Isso é chamado de jagged array (array irregular). É útil, por exemplo, quando estações diferentes registraram quantidades diferentes de leituras ao longo do dia, por causa de falhas pontuais de conexão.

```
// Jagged array – cada estação com um número diferente de leituras
double[][] leiturasIrregulares = new double[3][];
leiturasIrregulares[0] = new double[]{18.0}; // 1 leitura
leiturasIrregulares[1] = new double[]{20.0, 21.5}; // 2 leituras
leiturasIrregulares[2] = new double[]{16.0, 17.2, 18.9}; // 3 leituras

for (double[] estacao : leiturasIrregulares) {
    System.out.println(java.util.Arrays.toString(estacao));
}
// [18.0]
// [20.0, 21.5]
// [16.0, 17.2, 18.9]
```

Arrays Multidimensionais

Arrays 3D, 4D e quando (não) vale a pena usá-los

Arrays com mais de duas dimensões

O Java permite arrays com qualquer número de dimensões. Um array 3D, declarado como `double[][][]`, pode ser imaginado como um cubo de valores — por exemplo, estação × dia do mês × hora do dia. Na prática, dimensões acima de 3 são raras: a complexidade de gerenciar tantos índices costuma superar o benefício, e nesses casos listas de listas ou classes próprias tendem a ser mais legíveis.

Os usos mais comuns de arrays 3D incluem séries temporais organizadas por várias categorias simultâneas — como no exemplo da estação meteorológica — além de simulações físicas e problemas de programação dinâmica com três variáveis de estado.

```
// Array 3D: estação x dia x hora
double[][][] leiturasDoMes = new double[3][30][24]; // 3 estações, 30 dias, 24 horas

leiturasDoMes[0][14][8] = 21.5; // Estação 0, dia 15, 9h da manhã
System.out.println(leiturasDoMes[0][14][8]); // 21.5

// Arrays.deepToString imprime arrays multidimensionais de forma legível
import java.util.Arrays;
double[][] amostra = {{18.0, 19.0}, {20.0, 21.0}};
System.out.println(Arrays.deepToString(amostra)); // [[18.0, 19.0], [20.0, 21.0]]
```

DICA

Para imprimir arrays multidimensionais de forma legível, use `Arrays.deepToString(array)` em vez de `Arrays.toString(array)` — o segundo mostra apenas os endereços de memória dos arrays internos.

Resumo da Apostila

Os conceitos essenciais desta apostila, em uma única tabela de consulta

Conceito	O que significa
Array	Estrutura que guarda vários valores do mesmo tipo, acessados por índice a partir de 0.
new vs initializer	new aloca com valores padrão; o initializer ({...}) já define os valores na criação.
length	Campo (não método) que devolve o tamanho do array. Último índice válido é <code>length - 1</code> .
Array como objeto	Herda de Object. A variável guarda uma referência, não o array em si.
Cópia de array	<code>array2 = array1</code> compartilha o mesmo objeto. Use <code>Arrays.copyOf</code> para uma cópia independente.
For-each	Percorre elementos sem controlar índice, mas não permite modificar o array original.
<code>java.util.Arrays</code>	<code>sort</code> , <code>fill</code> , <code>copyOf</code> , <code>toString</code> e <code>equals</code> — métodos estáticos utilitários para arrays.
Busca linear	Percorre elemento a elemento. $O(n)$. Funciona em qualquer array.
Busca binária	$O(\log n)$, exige array ordenado. <code>Arrays.binarySearch</code> já implementa o algoritmo.
Varargs (...)	Permite métodos com número variável de argumentos, tratados como array internamente.
Array 2D	Array de arrays, com acesso via <code>[linha][coluna]</code> . Base para representar matrizes e grades.
Jagged array	Array 2D cujas linhas podem ter tamanhos diferentes entre si.
Arrays multidimensionais	Suportados além do 2D, mas usados com moderação por causa da complexidade.